Lab Assignment 7

📝 Aim

To understand and implement normalization techniques in relational databases by applying 1NF, 2NF, 3NF, and BCNF, ensuring data consistency, eliminating redundancy, and improving database efficiency.

🎯 Objectives

- To learn the concept of normalization and its importance in relational database design.
- To apply **1NF** for eliminating repeating groups and ensuring atomic values.
- To apply **2NF** for removing partial dependencies in composite keys.
- To apply **3NF** for eliminating transitive dependencies.
- To apply **BCNF** for achieving stricter normalization by removing anomalies not covered in 3NF.
- To gain hands-on experience in restructuring tables to achieve higher normal forms.

⚙️ Requirements

- **Software/Tools:**
  - MySQL / PostgreSQL / Oracle DB / SQL Server
  - SQL command-line client or GUI (e.g., MySQL Workbench, pgAdmin)
- **System Setup:**
  - Installed RDBMS with active database connection
- **Knowledge Prerequisites:**
  - Basic SQL syntax (CREATE, INSERT, SELECT)
  - Understanding of primary keys, foreign keys, and functional dependencies
  - Familiarity with relational database concepts

📖 Theory

🔹 First Normal Form (1NF)

- A relation is in 1NF if it contains only **atomic values** (no multi-valued or repeating groups).
- Each column must store a single value, and each record must be unique.
- Example: Splitting a column that stores multiple phone numbers into separate rows.

🔹 Second Normal Form (2NF)

- A relation is in 2NF if it is already in 1NF and **all non-key attributes are fully dependent on the primary key**.
- Removes **partial dependency** (when a non-key attribute depends only on part of a composite key).
- Example: In a table with composite key (StudentID, CourseID), attributes like StudentName should depend only on StudentID, not the composite key.

🔹 **Third Normal Form (3NF)**

- A relation is in 3NF if it is in 2NF and **no transitive dependency exists**.
- Transitive dependency occurs when a non-key attribute depends on another non-key attribute.
- Example: If StudentID → DepartmentID → DepartmentName, then DepartmentName should be placed in a separate table.

🔹 **Boyce-Codd Normal Form (BCNF)**

- A stricter version of 3NF.
- A relation is in BCNF if **for every functional dependency (X → Y), X is a superkey**.
- Eliminates anomalies that 3NF may not handle.
- Example: In a table where TeacherID → Subject and Subject → TeacherID, BCNF requires restructuring to avoid dependency conflicts.

Code:

1) CREATE TABLE StudentCourse (

StudentID INT,

StudentName VARCHAR(50),

Courses VARCHAR(100), -- multiple courses in one field

Department VARCHAR(50),

DeptHead VARCHAR(50)

);

INSERT INTO StudentCourse VALUES

(1, 'Amit', 'Math, Physics', 'Science', 'Dr. Rao'),

(2, 'Sneha', 'Chemistry', 'Science', 'Dr. Rao'),

(3, 'Raj', 'History, English', 'Arts', 'Dr. Mehta');

2) CREATE TABLE StudentCourse_1NF (

```
    StudentID INT,
    StudentName VARCHAR(50),
    Course VARCHAR(50),
    Department VARCHAR(50),
    DeptHead VARCHAR(50)
);

INSERT INTO StudentCourse_1NF VALUES
(1, 'Amit', 'Math', 'Science', 'Dr. Rao'),
(1, 'Amit', 'Physics', 'Science', 'Dr. Rao'),
(2, 'Sneha', 'Chemistry', 'Science', 'Dr. Rao'),
(3, 'Raj', 'History', 'Arts', 'Dr. Mehta'),
(3, 'Raj', 'English', 'Arts', 'Dr. Mehta');
```

3) CREATE TABLE Students (

```
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Department VARCHAR(50),
    DeptHead VARCHAR(50)
);

CREATE TABLE Courses (
    StudentID INT,
    Course VARCHAR(50),
    FOREIGN KEY (StudentID) REFERENCES Students(StudentID)
);

-- Insert data
INSERT INTO Students VALUES
```

```
(1, 'Amit', 'Science', 'Dr. Rao'),
(2, 'Sneha', 'Science', 'Dr. Rao'),
(3, 'Raj', 'Arts', 'Dr. Mehta');

INSERT INTO Courses VALUES
(1, 'Math'),
(1, 'Physics'),
(2, 'Chemistry'),
(3, 'History'),
(3, 'English');
```

4)
```
CREATE TABLE Department (
    Department VARCHAR(50) PRIMARY KEY,
    DeptHead VARCHAR(50)
);

-- Update Students table
CREATE TABLE Students_3NF (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    Department VARCHAR(50),
    FOREIGN KEY (Department) REFERENCES Department(Department)
);

-- Insert data
INSERT INTO Department VALUES
('Science', 'Dr. Rao'),
('Arts', 'Dr. Mehta');

INSERT INTO Students_3NF VALUES
```

```
(1, 'Amit', 'Science'),
(2, 'Sneha', 'Science'),
(3, 'Raj', 'Arts');

CREATE TABLE Department_BCNF (
    DeptHead VARCHAR(50) PRIMARY KEY,
    Department VARCHAR(50)
);

-- Students table now references DeptHead
5) CREATE TABLE Students_BCNF (
    StudentID INT PRIMARY KEY,
    StudentName VARCHAR(50),
    DeptHead VARCHAR(50),
    FOREIGN KEY (DeptHead) REFERENCES Department_BCNF(DeptHead)
);

-- Insert data
INSERT INTO Department_BCNF VALUES
('Dr. Rao', 'Science'),
('Dr. Mehta', 'Arts');

INSERT INTO Students_BCNF VALUES
(1, 'Amit', 'Dr. Rao'),
(2, 'Sneha', 'Dr. Rao'),
(3, 'Raj', 'Dr. Mehta');
```

Output:

Conclusion: